CS23532 – COMPUTER NETWORKS
EX 14: Packet Sniffing using Raw Sockets

AIM :

To write a Java program that uses a raw socket to capture and analyse the packets travelling through the network interface (packet sniffing).

ALGORITHM :

Java does not provide a raw socket class, so the program calls the operating system's socket() and recvfrom() functions directly using the Java Foreign Function and Memory API (Java 21). The program runs on Linux and needs root privileges.

1. Import the java.lang.foreign classes and create a native linker.

2. Create a raw packet socket using socket(AF_PACKET, SOCK_RAW, ETH_P_ALL) so that every frame reaching the interface is received.

3. Allocate a buffer for the incoming frames.

4. Receive a frame using recvfrom(). Ignore the copies of outgoing packets so that each packet is displayed only once.

5. Decode the Ethernet header: the destination MAC address, the source MAC address and the EtherType.

6. If the EtherType is 0x0800 (IPv4), decode the IP header: the source and destination IP addresses, the TTL and the protocol number.

7. Decode the protocol: for ICMP display the message type (Echo Request / Echo Reply); for TCP and UDP display the source and destination port numbers.

8. Repeat for the required number of packets and then stop.

PROGRAM :

PacketSniffer.java

```
import java.lang.foreign.*;
import java.lang.invoke.MethodHandle;

public class PacketSniffer {

    static int u8(MemorySegment b, int i) {
        return b.get(ValueLayout.JAVA_BYTE, i) & 0xFF;
    }

    static int u16(MemorySegment b, int i) {
        return (u8(b, i) << 8) | u8(b, i + 1);
```

```
    }

    static String mac(MemorySegment b, int i) {
        StringBuilder sb = new StringBuilder();
        for (int k = 0; k < 6; k++) {
            sb.append(String.format("%02x", u8(b, i + k)));
            if (k < 5) sb.append(":");
        }
        return sb.toString();
    }

    static String ip(MemorySegment b, int i) {
        return u8(b, i) + "." + u8(b, i + 1) + "." + u8(b, i + 2) + "." +
u8(b, i + 3);
    }

    public static void main(String[] args) throws Throwable {
        int maxPackets = args.length > 0 ? Integer.parseInt(args[0]) : 10;
        Linker linker = Linker.nativeLinker();
        SymbolLookup libc = linker.defaultLookup();
        MethodHandle socket =
linker.downcallHandle(libc.find("socket").get(),
                FunctionDescriptor.of(ValueLayout.JAVA_INT,
ValueLayout.JAVA_INT, ValueLayout.JAVA_INT, ValueLayout.JAVA_INT));
        MethodHandle recvfrom =
linker.downcallHandle(libc.find("recvfrom").get(),
                FunctionDescriptor.of(ValueLayout.JAVA_LONG,
ValueLayout.JAVA_INT, ValueLayout.ADDRESS, ValueLayout.JAVA_LONG,
                        ValueLayout.JAVA_INT, ValueLayout.ADDRESS,
ValueLayout.ADDRESS));

        int fd = (int) socket.invokeExact(17, 3, 0x0300);
        if (fd < 0) {
            System.out.println("Unable to create the raw socket. Run as
root.");
            return;
        }
        System.out.println("Raw socket created. Sniffing " + maxPackets + "
packets...\n");

        try (Arena arena = Arena.ofConfined()) {
            MemorySegment buf = arena.allocate(65536);
            MemorySegment addr = arena.allocate(32);
            MemorySegment addrLen = arena.allocate(4);
            int n = 0;
            while (n < maxPackets) {
                addrLen.set(ValueLayout.JAVA_INT, 0, 32);
                long len = (long) recvfrom.invokeExact(fd, buf, 65536L, 0,
addr, addrLen);
                if (u8(addr, 10) == 4) {
                    continue;
                }
                n++;
                int type = u16(buf, 12);
```

```
                System.out.println("Packet " + n + " (" + len + " bytes)");
                System.out.println("  Ethernet : " + mac(buf, 6) + " -> " +
mac(buf, 0));
                if (type != 0x0800) {
                    System.out.println("  EtherType: 0x" +
Integer.toHexString(type));
                    continue;
                }
                int ihl = (u8(buf, 14) & 0x0F) * 4;
                int ttl = u8(buf, 22);
                int proto = u8(buf, 23);
                String src = ip(buf, 26);
                String dst = ip(buf, 30);
                int t = 14 + ihl;
                if (proto == 1) {
                    int icmp = u8(buf, t);
                    String name = icmp == 8 ? "Echo Request" : icmp == 0 ?
"Echo Reply" : "Type " + icmp;
                    System.out.println("  IPv4     : " + src + " -> " + dst
+ "  TTL=" + ttl);
                    System.out.println("  Protocol : ICMP (" + name + ")");
                } else if (proto == 6 || proto == 17) {
                    System.out.println("  IPv4     : " + src + ":" +
u16(buf, t) + " -> " + dst + ":" + u16(buf, t + 2) + "  TTL=" + ttl);
                    System.out.println("  Protocol : " + (proto == 6 ?
"TCP" : "UDP"));
                } else {
                    System.out.println("  IPv4     : " + src + " -> " + dst
+ "  TTL=" + ttl);
                    System.out.println("  Protocol : " + proto);
                }
            }
        }
        System.out.println("\nSniffing completed.");
    }
}
```

INPUT AND OUTPUT :

Input: Compile and run the program as root and generate some network traffic (ping and UDP/TCP messages to localhost) while it is sniffing. The number of packets to capture is given as the argument (10).

```
javac --release 21 --enable-preview PacketSniffer.java
sudo java --enable-preview --enable-native-access=ALL-UNNAMED PacketSniffer
10
```

Output

```
Raw socket created. Sniffing 10 packets...


Packet 1 (74 bytes)
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
```

```
  IPv4      : 127.0.0.1 -> 127.0.0.1  TTL=64
  Protocol : ICMP (Echo Request)
Packet 2 (74 bytes)
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4      : 127.0.0.1 -> 127.0.0.1  TTL=64
  Protocol : ICMP (Echo Reply)
Packet 3 (74 bytes)
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4      : 127.0.0.1 -> 127.0.0.1  TTL=64
  Protocol : ICMP (Echo Request)
Packet 4 (74 bytes)
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4      : 127.0.0.1 -> 127.0.0.1  TTL=64
  Protocol : ICMP (Echo Reply)
Packet 5 (74 bytes)
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4      : 127.0.0.1 -> 127.0.0.1  TTL=64
  Protocol : ICMP (Echo Request)
Packet 6 (74 bytes)
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4      : 127.0.0.1 -> 127.0.0.1  TTL=64
  Protocol : ICMP (Echo Reply)
Packet 7 (74 bytes)
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4      : 127.0.0.1 -> 127.0.0.1  TTL=64
  Protocol : ICMP (Echo Request)
Packet 8 (74 bytes)
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4      : 127.0.0.1 -> 127.0.0.1  TTL=64
  Protocol : ICMP (Echo Reply)
Packet 9 (47 bytes)
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4      : 127.0.0.1:40164 -> 127.0.0.1:6000  TTL=64
  Protocol : UDP
Packet 10 (75 bytes)
  Ethernet : 00:00:00:00:00:00 -> 00:00:00:00:00:00
  IPv4      : 127.0.0.1 -> 127.0.0.1  TTL=64
  Protocol : ICMP (Type 3)

Sniffing completed.
```

RESULT:

Thus, a Java program using a raw socket was written to capture packets, and the Ethernet, IP and TCP/UDP/ICMP header details of the captured packets were displayed successfully.